



A PYTHON TRIAL OF THE OLD CODE

FANTASY BATTLE ARENA

*In the realm of Mythoria, warriors, mages, and archers clash for
supremacy — and you must forge the very laws of their combat.*



DIFFICULTY · INTERMEDIATE

PYTHON 3

OBJECT-ORIENTED PROGRAMMING

INHERITANCE

POLYMORPHISM

PRACTICE WORKSHEET · CLASS DESIGN & INHERITANCE

I THE QUEST

In the world of **Mythoria**, legendary fighters battle for ultimate supremacy. Every champion carries their own strengths, weaknesses, and secret arts. As the newly-appointed Arena Architect, the High Council has handed you a sacred duty: build the combat engine that governs every duel in the arena.

Your mission is to design a **turn-based combat system** in Python where three classes of fighter — the Warrior, the Mage, and the Archer — do battle until only one remains standing. Rather than copy-pasting the same code for each fighter, you will use **inheritance** to write shared behaviour once and let each class specialise it.

This is a design problem as much as a coding problem. Before you write a single line, think about what *every* fighter has in common, and what makes each one unique. The common parts belong in a parent class; the unique parts belong in the children. Getting that split right is the heart of the challenge.

II WHAT YOU WILL PRACTISE



INHERITANCE

Derive Warrior, Mage & Archer from a single Character base class so shared logic lives in one place.



METHOD OVERRIDING

Let each subclass redefine attack() and special moves to express its own fighting style.



POLYMORPHISM

Call the same attack() on any fighter and watch each respond in its own way.



ENCAPSULATION

Keep each fighter's state (health, mana, rage) bundled with the methods that change it.



STATE & LOGIC

Model changing values across turns and translate combat rules into clean conditionals.



CLASS DESIGN

Decide what is shared vs. unique, and place attributes & methods where they belong.

III THE CODEX OF CONCEPTS

Before entering the arena, learn the language of the Old Code. These are the OOP ideas this challenge is built on — read them once, and the task ahead will read like a story.

CLASS `class Warrior:`

A **blueprint** that describes a kind of thing — what data it holds and what it can do. *Think of it as the design of a fighter, not a fighter itself.*

OBJECT / INSTANCE `thor = Warrior(...)`

A concrete thing **built from a class**. Each object has its own copy of the data. *Thor and Ragnar are two separate objects made from the same Warrior blueprint, each with their own health.*

ATTRIBUTE `self.health = 120`

A **variable that belongs to an object** and stores its current state — its name, health, attack power, and so on. Attributes are the fighter's "stats".

METHOD `def attack(self, target):`

A **function defined inside a class** that describes what an object can *do*. Methods act on the object's own attributes through `self`.

INHERITANCE `class Mage(Character):`

A mechanism where a **child class** automatically gains the attributes and methods of a **parent class**. Write shared behaviour once in the parent; every child receives it for free. *All fighters can take damage — so `take_damage()` lives in the parent.*

SUPERCLASS & SUBCLASS `Character → Warrior`

The **superclass** (parent / base) is the class being inherited from. The **subclass** (child / derived) is the class that inherits. Here, `Character` is the superclass; `Warrior`, `Mage`, and `Archer` are its subclasses.

SUPER() `super().__init__()`

A built-in function that lets a subclass **call the parent's methods** — most often the parent's constructor — so it can reuse the parent's setup instead of rewriting it. *A Warrior sets up name/health via the parent, then adds its own rage.*

METHOD OVERRIDING `redefine attack() in a child`

When a subclass **redefines an inherited method** to give it specialised behaviour. Same method name, different implementation — this is how each fighter attacks in its own signature style.

POLYMORPHISM `for f in fighters: f.attack(t)`

"Many forms" — the ability to call **the same method name on different object types** and have each respond appropriately. Your battle loop can treat every fighter uniformly and still get Warrior, Mage, and Archer behaviour.

ENCAPSULATION `bundle data + behaviour`

Keeping an object's **data and the methods that change it together** inside one class, and controlling how that data is accessed. A fighter's health should only change through methods like `take_damage()`, never by reaching in from outside at random.

TV THE FOUNDATION — `class Character`

Every fighter in Mythoria is, at heart, a **Character**. This is your **superclass** — the single blueprint that all three fighter types inherit from. Define the shared stats and shared actions here *once*, and the Warrior, Mage, and Archer will build on top of it.

ATTRIBUTES — THE SHARED STATS

ATTRIBUTE	TYPE	MEANING
<code>name</code>	str	The fighter's name, used in the battle log.
<code>health</code>	int	Current health points. The fighter falls when this reaches 0 or below .
<code>max_health</code>	int	Starting health — remember it, so you can compute percentages (needed for Berserk).
<code>attack_power</code>	int	Base strength of an ordinary attack.
<code>defense</code>	int	Reduces incoming damage before it hits health.
<code>speed</code>	int	Determines who acts first each round (higher goes first).

METHODS — THE SHARED ACTIONS

METHOD	WHAT IT MUST DO
<code>__init__(self, ...)</code>	The constructor. Accept and store all of the attributes above when a fighter is created.
<code>attack(self, target)</code>	Compute damage against a target and apply it. This is the method each subclass will override .
<code>take_damage(self, amount)</code>	Subtract the fighter's <code>defense</code> from <code>amount</code> (minimum 1) and reduce <code>health</code> accordingly.
<code>is_alive(self)</code>	Return <code>True</code> while <code>health > 0</code> , otherwise <code>False</code> .

```
# The one damage rule every attack obeys
final_damage = max(1, attacker.attack_power - target.defense)
# Defense can soften a blow, but never fully stop it — a hit always
costs at least 1 HP.
```

V THE THREE CHAMPIONS

Each champion **inherits** from Character, adds its own extra attribute, and **overrides** combat to unleash a signature ability. Suggested starting stats are given as chips — tune them if you wish, but keep the classes balanced.



WARRIOR

MELEE BRUISER
HIGH HEALTH · HIGH DEFENSE

A fearless close-combat fighter who grows deadlier the closer they are to death.

health 130

attack 22

defense 12

speed 6

+ rage 0

New attribute — **rage**: builds up each time the Warrior takes damage. You may use it to add bonus attack power once it is "full" (design your own threshold).



BERSERK MODE

When the Warrior's **health** drops **below 30% of max_health**, their **attack_power** is **doubled** for their attacks. Announce it in the log the moment it triggers.



MAGE

SPELLCASTER
HIGH ATTACK · LOW DEFENSE

A master of raw magic who trades safety and vitality for devastating power.

health 90

attack 30

defense 5

speed 8

+ mana 100

New attribute — **mana**: the fuel for spells. When mana is too low to cast, the Mage must fall back on an ordinary attack.



FIREBALL

Costs **mana** to cast and deals **massive damage** (e.g. $\sim 1.5 \times$ attack power) — but the backlash **also harms the Mage** slightly. Cast only when enough mana remains; otherwise attack normally.



ARCHER

RANGED MARKSMAN
HIGH SPEED · CRITICAL HITS

A precise sharpshooter who strikes first and, now and then, strikes twice as hard.

health 100

attack 24

defense 7

speed 12

+ critical_chance 0.30

New attribute — **critical_chance**: the probability (0.0–1.0) of landing a critical strike on any given attack.



PRECISION SHOT

On each attack there is a **30% chance** (use **random**) to land a **Critical Hit** that deals **2× damage**. Announce criticals distinctly in the log.

VI LAWS OF THE ARENA

TURN ORDER

-  Combat is **turn-based**: each round, fighters act one at a time.
-  The fighter with the **highest speed** attacks **first** each round.
-  If two fighters have **equal speed**, the **player-controlled** character acts first.
-  A fallen fighter (health ≤ 0) is skipped and takes no further turns.

DAMAGE CALCULATION

```
# Step 1 – base damage from the attacker
raw = attacker.attack_power

# Step 2 – special abilities may change it
Warrior → if health < 30% max: raw = raw × 2 # Berserk
Mage → Fireball: raw = raw × 1.5 (costs mana, self-damage)
Archer → 30% chance: raw = raw × 2 # Critical

# Step 3 – defense soaks some of it, but never all
final_damage = max(1, raw - target.defense)
```

VICTORY CONDITION

-  A fighter is **defeated** the instant their health falls to **0 or below**.
-  The battle **ends when only one fighter remains standing**.
-  That last survivor is declared the **champion of the arena**.

VII A BATTLE FORETOLD

Your finished program should narrate the fight turn by turn. Here is the kind of log the arena expects — your wording may differ, but the *events* should read like this:

```
🏹 Alex (Archer) shoots an arrow! Deals 22 damage.
⚔️ Thor (Warrior) swings a sword! Deals 25 damage.
🔥 Gandalf (Mage) casts Fireball! Deals 40 damage but loses 5 health.
💥 Thor (Warrior) enters Berserk Mode! Attack power increased.
⚔️ Thor (Warrior) strikes with double power! Deals 50 damage.
🎯 Alex (Archer) lands a Critical Hit! Deals 44 damage.
💀 Thor (Warrior) is defeated!
🎊 Alex (Archer) wins the battle!
```

✦ YOUR TASK — DELIVERABLES

Write a single Python program that satisfies every point below:

- ✓ Define a **Character** superclass with the shared attributes and the four methods (**attack**, **take_damage**, **is_alive**, plus **__init__**).
- ✓ Use **inheritance** to build **Warrior**, **Mage**, and **Archer** from **Character**, each calling **super().__init__()** and adding its own attribute.
- ✓ **Override** the combat behaviour in each subclass to implement its **special ability** (Berserk, Fireball, Precision Shot).
- ✓ Simulate a **turn-based battle** that respects **speed** for turn order and applies the **damage formula** correctly.
- ✓ Ensure **defense**, **speed**, and **health** all genuinely affect the outcome, and **print a readable battle log** ending in a single champion.

✦ ARCHITECT'S GUIDANCE

You are not given the answer — but here is the shape of a good solution to keep you moving.

```
# A skeleton to grow from — fill in the logic yourself
import random

class Character:
    def __init__(self, name, health, attack_power, defense, speed):
        self.name = name
        self.health = health
        self.max_health = health      # remember the starting value
        self.attack_power = attack_power
        self.defense = defense
        self.speed = speed

    def take_damage(self, amount):
        dmg = max(1, amount - self.defense)
        self.health -= dmg
        return dmg

    def is_alive(self):
        return self.health > 0

    def attack(self, target):
        ...    # base attack — subclasses will override this

class Warrior(Character):
    def __init__(self, name, ...):
        super().__init__(name, ...)    # reuse the parent's setup
        self.rage = 0
    def attack(self, target):
        ...    # check Berserk, then apply the damage formula
```



REUSE, DON'T REPEAT

If you're writing the same code in two subclasses, it probably belongs in **Character**. Inheritance rewards you for being lazy in the right way.



RANDOMNESS FOR CRITS

Use **random.random() < 0.30** to model the Archer's 30% critical chance cleanly.



PERCENTAGES NEED MAX_HEALTH

Berserk triggers below 30% health — that's **health < 0.30 * max_health**, so store max_health at birth.



ONE LOOP, MANY TYPES

Sort fighters by **speed** and loop, calling **fighter.attack(target)** — polymorphism does the rest.

✦ ★ BONUS QUESTS

Finished early, or hungry for glory? These stretch goals push your design further — and prove that a well-built class hierarchy is easy to extend.



FULL 3-WAY FREE-FOR-ALL

Let all three fighters battle at once — remove the fallen after each turn and continue until a single survivor remains.



RAGE PAYOFF

Actually spend the Warrior's rage: once it crosses a threshold you choose, unleash a one-off bonus-damage strike and reset it.



ADD A FOURTH CLASS

Design a brand-new fighter — a Healer that restores HP, or a Rogue that dodges — inheriting from Character. If it slots in cleanly, your design is truly extensible.



GUARD THE GATES (VALIDATION)

Use encapsulation to protect each fighter's stats: never let health drop below 0 or mana go negative, and reject impossible values at creation.



A FAIRER DUEL (TESTING)

Run the same match-up many times and count the wins. Are the three classes balanced? Tune the starting stats until no class dominates.



ENTER THE ARENA

The blueprint is written; the fighters await your code. Design the classes, honour the rules, and let the simulation decide who walks out of Mythoria a legend.